

03. Evaluation Metrics Specification

Dynamic Workload Allocation Between Space and Ground

Team [Pas encoe]

Version 2.0 - September 2026

Table des matières

1	The Primary Endpoint : Processing Time (M_T)	2
1.1	Special Case : M_T with Task Dependencies (PSPLIB)	2
2	Diagnostic Metrics (The Four Pillars)	2
2.1	M_L (Load Balance)	2
2.2	M_Q (Queue Instability Index)	3
2.3	M_R (Resilience)	4
2.4	M_E (Efficiency)	4
3	Comparison Protocol (How Strategies Are Compared Fairly)	5
3.1	The Shared Surface Score	5
3.2	The Silent-Failure Hypothesis (Falsifiable)	5
3.3	Pareto-Optimality Analysis (The Trade-off Visualization)	6
4	Variable Dependency Graph (Declared)	7
4.1	M_S (Decision Stability)	7
4.2	Declared Dependencies	8

1 The Primary Endpoint : Processing Time (M_T)

The primary comparison metric for the overall success of a strategy is the total Makespan.

$$M_T = \max_{task \in T} (\text{CompletionTime}_{task}) - \min_{task \in T} (\text{ArrivalTime}_{task}) \quad (1)$$

Implementation : Extracted from the simulation timeline.

Interpretation : Lower is better. A strategy that quickly clears the queue (Strategies 3–5) will outperform baselines (Strategies 1/2) here.

C'est le temps total de la mission, depuis l'arrivée de la première tâche jusqu'à la fin de la dernière tâche.

La formule en langage humain

$$M_T = t_{\text{fin de la dernière tâche}} - t_{\text{arrivée de la première tâche}} \quad (2)$$

L'exemple du restaurant (très simple) 12h00 : Le premier client arrive et commande (première tâche).

12h30 : Les clients suivants arrivent.

13h00 : Le dernier client (la 150ème tâche) est enfin servi et a fini de manger.

Calcul : 13h00 - 12h00 = 1 heure.

Cette heure, c'est votre M_T (le "Makespan").

1.1 Special Case : M_T with Task Dependencies (PSPLIB)

When evaluating on PSPLIB instances, the standard makespan formula remains valid :

$$M_T = \max(\text{CompletionTime}) - \min(\text{ArrivalTime}).$$

However, the orchestrator must enforce **precedence constraints** : a task cannot start until all its predecessors have reached the COMPLETED state.

Impact on calculation : The completion time of the final task is naturally delayed by the critical path of the dependency graph. Our metrics (M_T , M_L , M_E) are computed identically, but their interpretation is adjusted to acknowledge that dependencies are an external constraint, not a scheduling artifact. This is explicitly reported in the Phase 3 results appendix.

2 Diagnostic Metrics (The Four Pillars)

To understand *why* a strategy performs well, we evaluate it against the constraints defined in the IASTAM spec.

2.1 M_L (Load Balance)

Concept : Measures how evenly the computational load is distributed across available nodes. High variance indicates a bottleneck.

Formula : Compute CPU utilization (ρ_i) for each node i over the simulation period.

$$M_L = \sigma(\rho_1, \rho_2, \dots, \rho_n) \quad (3)$$

(Standard Deviation of the utilization rates).

Interpretation : Lower is better. A score of 0 means perfect balance. High values mean a single node was overworked.

C'est une mesure de justice entre vos machines (satellites, stations, cloud). Cela vérifie si vous avez bien réparti le travail, ou si vous avez surchargé une seule machine alors que les autres ne font rien.

L'analogie de la cuisine (très simple) Imaginez une cuisine avec 4 cuisiniers (vos nœuds).

Situation parfaite ($ML = 0$) : Chaque cuisinier prépare exactement 25 plats. Ils sont tous aussi occupés les uns que les autres. Tout roule.

Situation pourrie (ML très élevé) : Le Cuisinier 1 doit préparer 90 plats tout seul (il est en sueur, il ralentit tout le service). Les Cuisiniers 2, 3 et 4 s'ennuient et ne font que 3 plats chacun.

Le calcul : On regarde l'écart entre le plus occupé et le moins occupé. Plus l'écart est grand, plus le ML est grand.

2.2 M_Q (Queue Instability Index)

Concept : Measures how often a strategy allows a node to enter a critical saturation zone, where $\rho > 0.85$. This threshold is derived from queueing theory, where waiting times grow asymptotically as $\rho \rightarrow 1$.

Formula :

$$M_Q = \frac{1}{T} \int_0^T \max(0, \rho_i(t) - 0.85) dt \quad (4)$$

Interpretation : Lower is better. An M_Q close to 0 indicates that the strategy maintains healthy queue lengths. Strategy 2 (Always Orbital) is expected to have a high M_Q . Strategy 5 (RL) is expected to achieve the lowest M_Q by learning to anticipate bursts.

C'est une mesure du stress chronique des machines. Elle mesure combien de temps une machine passe dans la zone rouge, c'est-à-dire avec une utilisation supérieure à 85%, lorsque les files d'attente commencent à croître rapidement.

L'analogie du supermarché (très simple) Imaginez une caisse de supermarché (votre satellite).

Zone verte (saine) : Il y a 1 ou 2 clients dans la file. Le caissier travaille à 70%. Tout va bien.

Zone rouge (critique, $\rho > 0.85$) : Il y a 20 clients dans la file. Le caissier est occupé à 95%. La file ne désemplit pas et les clients attendent 30 minutes.

Le calcul (M_Q) On chronomètre le temps total passé en zone rouge.

Si le caissier passe 10 minutes en zone rouge, M_Q est moyen.

S'il passe 2 heures en zone rouge, M_Q est très élevé.

2.3 M_R (Resilience)

Concept : Measures the system's ability to recover from disruptions (link drops, node failure).

Implementation : We inject a failure event at a specific timestamp.

$$M_R = \frac{\text{Tasks Successfully Completed After Failure}}{\text{Total Orphaned Tasks at Failure Time}} \quad (5)$$

Extension : In the radiation-based profile, M_R distinguishes between :

- **Soft failures (SDC) :** Task is re-executed. Counts as a "recovery" if the second attempt succeeds.
- **Hard failures (HBM/SEFI) :** Node becomes unavailable. Counts as a "recovery" only if Strategies 4 or 5 successfully migrate orphaned tasks.

Interpretation : 1.0 is perfect (all tasks re-allocated). 0.0 means mission failure for those tasks. Strategies capable of re-scheduling or learning recovery policies (Strategies 4 and 5) can score high here. Strategies 1–3 will score 0 or very low.

C'est une mesure de la capacité de survie du système. Elle vérifie que, lorsqu'une machine tombe en panne ou qu'un lien satellite se coupe, le système peut sauver les tâches en cours et les relancer ailleurs au lieu de les perdre définitivement.

L'analogie du livreur en panne (très simple) Imaginez une flotte de livreurs (les satellites) qui transportent des colis (les tâches).

Panne ($t = 500$ s) : Le Livreur 1 (LEO-1) crève un pneu alors qu'il transporte 10 colis non livrés, qui deviennent des tâches orphelines.

Système non résilient ($M_R = 0$) : Les colis sont abandonnés et ne seront jamais livrés. La mission échoue pour ces tâches.

Système résilient ($M_R = 1$) : L'orchestrateur détecte le problème et réaffecte les 10 colis au Livreur 2. Tous les colis sont finalement livrés.

Calcul Pour le système résilient :

$$M_R = \frac{10 \text{ tâches récupérées}}{10 \text{ tâches orphelines}} = 1.0 \quad (6)$$

Pour le système non résilient :

$$M_R = \frac{0 \text{ tâche récupérée}}{10 \text{ tâches orphelines}} = 0.0 \quad (7)$$

2.4 M_E (Efficiency)

Concept : Resource expenditure per unit of useful work. Combines energy and bandwidth.

Formula :

$$M_E = \frac{\sum_{task} (\text{Energy}_{task} + \text{TransmissionCost}_{task})}{\text{Total Tasks Completed}} \quad (8)$$

Energy is defined per node (Section 2, Table 1 of 02_EXPERIMENTAL_DESIGN.md). *TransmissionCost* is proportional to the data size transmitted over the link.

Interpretation : Lower is better. Strategy 1 (Always Cloud) will have high transmission costs. Strategy 2 (Always Orbital) will have high energy costs. Strategies 4 and 5

aim to minimize the sum.

Qu'est-ce que c'est ? Il s'agit du coût moyen par tâche accomplie. Cette métrique additionne l'énergie dépensée pour le calcul et le coût réseau nécessaire à la transmission des données.

L'analogie de la livraison de pizzas (très simple) Imaginez un service qui doit livrer 100 pizzas, représentant les tâches à exécuter.

- **Faire la pizza** représente le calcul et sa consommation énergétique.
- **Conduire pour la livrer** représente la transmission et son coût réseau.
- **Stratégie 1 (Tout au Cloud)** : le coût de calcul local est faible, mais les nombreux transferts produisent un coût de transmission élevé.
- **Stratégie 2 (Tout en Orbite)** : les transferts sont limités, mais le calcul orbital consomme davantage d'énergie.
- **Stratégie 5 (Intelligente)** : l'agent recherche un compromis entre calcul local et transmission afin de réduire le coût moyen total.

Le calcul en langage humain

$$M_E = \frac{\text{Coût énergétique} + \text{Coût de transmission}}{\text{Nombre total de tâches terminées}} \quad (9)$$

Il s'agit donc d'une moyenne : le coût total par tâche terminée.

3 Comparison Protocol (How Strategies Are Compared Fairly)

To rank the strategies, we use a **Weighted Composite Score** derived from the mission specifications. The jury expects us to optimize for multiple constraints.

3.1 The Shared Surface Score

For primary between-strategy comparison, we normalize each metric M_i to a $[0, 1]$ scale relative to the *worst-case baseline* observed in Phase 1 (Strategy 1 or 2). Then we compute :

$$S_{\text{Composite}} = \frac{1}{4} \left(\frac{M_T^{\max} - M_T}{M_T^{\max} - M_T^{\min}} \right) + \frac{1}{4} M_L^{\text{norm}} + \frac{1}{4} M_R + \frac{1}{4} M_E^{\text{norm}} \quad (10)$$

(Note : For the first three terms, lower is better. For Resilience, higher is better. All components are min-max normalized to ensure equal weighting).

3.2 The Silent-Failure Hypothesis (Falsifiable)

A key question is : *Does the Greedy Heuristic (Strategy 3) appear efficient because it ignores long-term resilience ?* We test this by running Strategy 3 and Strategy 4 on the "Intermittent Connectivity" profile. If Strategy 3 achieves a good M_T but a catastrophic M_R , it proves the necessity of the proactive monitoring layer (Strategy 4). If Strategy 4 achieves a similar M_T but a significantly higher M_R , it validates our research hypothesis.

Il s'agit d'une expérience scientifique destinée à déterminer si une stratégie est réellement robuste ou si ses bonnes performances cachent une faiblesse.

Une stratégie peut sembler très efficace parce qu'elle termine rapidement les tâches, tout en restant incapable de gérer un événement imprévu.

L'analogie du conducteur pressé (très simple) Deux chauffeurs de bus doivent effectuer le même trajet, qui représente l'exécution des tâches.

Chauffeur 3 (Stratégie 3) : Il roule très vite et obtient un excellent M_T , mais ne surveille pas l'état du véhicule. Si le moteur tombe en panne près de l'arrivée, les passagers ne peuvent pas terminer le trajet. La stratégie était rapide, mais fragile.

Chauffeur 4 (Stratégie 4) : Il roule légèrement moins vite parce qu'il surveille régulièrement le véhicule. Lorsqu'une panne est détectée, il transfère les passagers dans un autre bus. Le temps total M_T est légèrement plus élevé, mais la résilience atteint $M_R = 1.0$ si tous les passagers arrivent.

Ce que cela signifie pour notre projet Le profil *Intermittent Connectivity* simule une panne réseau à $t = 500$ s, lorsque LEO-1 perd sa connexion.

Stratégie 3 (Greedy) : Elle n'a pas de mécanisme de secours. Les tâches terminées avant la panne bénéficient d'un temps faible, mais les tâches en cours sur LEO-1 sont perdues :

$$M_T \text{ faible,} \quad M_R = 0. \quad (11)$$

Stratégie 4 (Resilient) : Elle détecte les tâches orphelines et les réaffecte à LEO-2 ou au Cloud. La surveillance ajoute un léger délai, mais permet de sauver la mission :

$$M_T \text{ légèrement plus élevé,} \quad M_R \approx 1.0. \quad (12)$$

3.3 Pareto-Optimality Analysis (The Trade-off Visualization)

Since no single strategy can simultaneously minimize makespan (M_T) and energy consumption (M_E), we reject the idea of relying exclusively on a single ranking. Instead, for the final pitch, we will plot a **Pareto Front** for all five strategies across all workload profiles.

A strategy S_1 *dominates* S_2 if :

$$M_T(S_1) \leq M_T(S_2) \quad \text{and} \quad M_E(S_1) \leq M_E(S_2) \quad (13)$$

with at least one strict inequality. Strategies lying on the nondominated lower-left boundary of this trade-off space are considered Pareto-optimal.

Expected outcome : We hypothesize that Strategy 5 (RL) will dominate Strategy 3 (Greedy), while Strategies 1 and 2 will lie near opposite extremes of the trade-off space. This hypothesis will be accepted or rejected based on the experimental results.

Le front de Pareto permet de visualiser les compromis. Gagner en rapidité peut augmenter la consommation énergétique, tandis qu'économiser de l'énergie peut augmenter le temps d'exécution. Une solution appartient au front lorsqu'aucune autre solution n'est meilleure sur les deux métriques simultanément.

L’analogie du choix d’un véhicule (très simple) Pour aller au travail, nous cherchons à minimiser deux objectifs :

- le temps de trajet, qui représente M_T ;
- le coût du trajet, qui représente M_E .

Considérons trois possibilités :

- **La Formule 1** : 5 minutes, mais une consommation très élevée (M_T faible et M_E élevé).
- **La marche** : 2 heures, sans coût énergétique externe (M_T élevé et $M_E = 0$).
- **Le vélo** : 25 minutes, avec le même coût externe négligeable que la marche (M_T plus faible et $M_E = 0$).

Sur le graphique, la région inférieure gauche correspond aux solutions rapides et économes. La Formule 1 et le vélo représentent deux compromis différents : aucune des deux solutions ne domine l’autre. En revanche, le vélo domine la marche, car il possède le même coût tout en étant plus rapide.

Ce que cela signifie pour notre projet Il n’existe pas nécessairement de stratégie meilleure dans l’absolu. La Stratégie 1 peut obtenir un M_T faible avec un M_E élevé, tandis que la Stratégie 2 peut présenter le compromis inverse. L’objectif est de tracer les cinq stratégies et de déterminer empiriquement si la Stratégie 5 rejoint ou améliore le front de Pareto.

L’hypothèse (ce que vous pensez voir) : L’agent RL (Stratégie 5) devrait se trouver sur la courbe du Front de Pareto du côté du vélo : il n’est peut-être pas aussi rapide que la Stratégie 1, ni aussi économe que la Stratégie 2, mais il est le meilleur compromis global (il n’est dominé par aucune autre stratégie).

Si l’IA se retrouve en dessous de la courbe (dominée par les autres), votre hypothèse est fausse, mais ça reste un résultat scientifique intéressant à montrer au jury !

4 Variable Dependency Graph (Declared)

The six metrics are designed to capture complementary aspects of the system. However, they are not independent in a physics context.

4.1 M_S (Decision Stability)

Concept : We introduce this diagnostic metric to penalize oscillating strategies. A strategy that constantly reassigns tasks (e.g., due to nervous behavior in RL) wastes energy on network handovers.

Formula :

$$M_S = \frac{\text{Number of Task Migrations}}{\text{Total Completed Tasks}} \quad (14)$$

Interpretation : Lower is better. An M_S near 0 indicates a stable, confident scheduler. This metric is crucial to distinguish Strategy 4 (stable rules) from a poorly trained Strategy 5 (erratic behavior).

Cette métrique mesure la stabilité des décisions. Elle indique si l’orchestrateur réaffecte inutilement les tâches d’une machine à une autre.

L’analogie du manager indécis (très simple) Imaginez un manager qui doit répartir des dossiers entre ses employés, lesquels représentent les nœuds de calcul.

Manager stable ($M_S \approx 0$) : Il confie un dossier à l’employé A et lui permet de terminer le travail sans interruption.

Manager instable (M_S élevé) : Il transfère continuellement le même dossier entre plusieurs employés. Le temps et l’énergie sont alors consacrés aux transferts plutôt qu’au travail utile.

Exemples de calcul Si 5 migrations sont observées pour 100 tâches terminées :

$$M_S = \frac{5}{100} = 0.05. \quad (15)$$

Si 50 migrations sont observées pour 100 tâches terminées :

$$M_S = \frac{50}{100} = 0.5. \quad (16)$$

Utilité de la métrique

- **Diagnostiquer une IA mal entraînée** : un agent RL instable peut changer fréquemment de décision sans améliorer les performances.
- **Vérifier la robustesse des règles** : un M_S faible permet de montrer que la Stratégie 4 reste prévisible et évite les migrations inutiles.

Pourquoi M_S n’entre pas dans $S_{\text{Composite}}$ Deux raisons justifient ce choix :

1. M_S est une métrique de diagnostic, tandis que les quatre métriques principales M_T , M_L , M_R et M_E représentent directement les objectifs de mission.
2. Une migration consomme déjà de l’énergie et de la bande passante, donc son coût est inclus dans M_E . Ajouter également M_S au score composite produirait un double comptage.

Nous conservons donc M_S comme preuve complémentaire de la stabilité interne de la stratégie.

4.2 Declared Dependencies

- M_T (Makespan) $\xleftarrow{\text{(inverse relationship)}} M_E$ (Efficiency) (Faster often costs more energy)
- M_L (Load Balance) $\xrightarrow{\text{(improves)}} M_T$ (Makespan) (Sharing load reduces overall time)
- M_Q (Queue Instability) $\xrightarrow{\text{(increases)}} M_T$ (Saturation increases waiting time)
- M_R (Resilience) $\xrightarrow{\text{(costs overhead)}} M_T$ (Checking health and migrating takes time)
- M_S (Decision Stability) $\xrightarrow{\text{(reduces overhead)}} M_E$ (Fewer migrations usually consume less energy)

Consequences for analysis :

1. We will never interpret M_T and M_E as two independent confirmations of success ; we will present the **Pareto Frontier** (trade-off graph) in our final pitch.
2. The comparison protocol (Section 3.1) weights them equally to force the algorithm to find a compromise, rather than gaming a single metric.